Linux’ta Süreç ve Kaynak İzleme

Süreçleri gözlemleme ve CPU, bellek, I/O kullanımı

Süreçler hangi kaynakları paylaşır?

CPU	İşlemci zamanı sınırlıdır; hazır süreçler bu zamanı kullanmak için yarışır.
BELLEK	Çalışan süreçlerin kodu ve verisi bellekte yer kaplar; ihtiyaç ile kullanılabilir kapasite birlikte düşünülür.
G/Ç	Disk ve diğer giriş/çıkış aygıtları istekleri karşılar; bekleyen süreç CPU kullanmıyor olabilir.
ÇEKİRDEK	Kaynakları süreçler arasında yönetir. Kendisi de süreç oluşturma ve süreçler arası iletişim gibi işler için kullanılan yazılım kaynağıdır.

Bu haftanın içeriği

01 · SÜREÇ	ps/top/lsof ile süreçleri ve açık kaynakları incele
02 · İZ SÜRME	strace/ltrace sistem ve kütüphane çağrılarını gösterir
03 · THREAD	PID/TID ilişkisini ve paylaşılan kaynakları açıkla
04 · CPU	time, nice/renice ve load average ile CPU kullanımını yorumla

Program, proses ve iş

PROGRAM	Diskteki çalıştırılabilir dosya veya komut tanımıdır.
PROSES	Programın çalışan örneği; kendi PID ve çalışma bağlamına sahiptir.
İŞ (JOB)	Kabukta başlatılan komut veya komut hattının kabukça izlenen iş birimidir.
İLİŞKİ	Bir kabuk işi bir veya birden fazla prosesten oluşabilir. İş numarası PID değildir.

PID, PPID ve proses ağacı

PID	Çalışan prosese sistemin verdiği kimliktir.
PPID	Ebeveyn proses kimliğidir; başlatma ilişkisini anlamaya yardım eder.
AĞAÇ	Kabuk bir komut başlatabilir; o komut başka prosesler de oluşturabilir.
SINIR	PID zamanla yeniden kullanılabilir. Tek başına kalıcı uygulama kimliği değildir.

Linux dağıtımı ve oturum bilgileri

```
uname -r
cat /etc/os-release
echo "$SHELL"
whoami
```

uname	Çekirdek sürümünü gösterir; dağıtım adı değildir.
os-release	Dağıtım ve sürüm bilgisini verir.
SHELL	Oturum için ayarlı kabuk yolunu gösterir; şu an kullanılan tüm alt kabukları saymaz.
whoami	Geçerli kullanıcı adını gösterir.

PID ve ebeveyn PID’yi gör

```
sleep 600 &
lesson_pid=$!
printf "PID=%s
" "$lesson_pid"
ps -o pid,ppid,stat,comm -p "$lesson_pid"
```

sleep	Zamanlı, sınırlı bir örnek iş başlatır; CPU yükü üretmez.
$!	Bu kabukta en son arka planda başlatılan işin proses kimliğidir.
ps -p	Belirtilen PID için seçilen alanları gösterir.
DİKKAT	Çıktıdaki PID örnek değildir, sizin oturumunuzda atanır. İş daha önce bittiyse görünmeyebilir.

ps komutu

ps -ef	Geniş proses listesini gösterir; seçenek biçimi geleneksel UNIX tarzıdır.
ps aux	BSD tarzı yaygın listedir. Seçenek yazımı ps -ef ile aynı kurala dayanmaz.
ps -u "$USER"	Kullanıcıya göre süreçleri daraltabilir. Sisteme göre kullanıcı biçimi farklı yorumlanabilir.
SEÇİM	Önce hangi soruyu sorduğunuzu belirleyin: kime ait, hangi PID, hangi ebeveyn?

ps parametreleri

```
ps -u "$(id -u)" -o pid,ppid,stat,%cpu,%mem,etime,comm
```

PID / PPID	Proses ve ebeveyn kimliği.
STAT	Linux proses durum kodları ve ek işaretler.
%CPU / %MEM	ps uygulamasının hesapladığı kaynak özetleri.
ETIME / COMM	Geçen süre ve komut adı. Komut adı her zaman tam komut satırı değildir.

STAT çıktısını dikkatli okuyun

R	Çalışıyor veya çalışmaya hazır. Yalnızca bu harf, CPU'yu o anda kullandığını kanıtlamaz.
S	Kesilebilir uyku: genellikle bir olayı bekleyen durum.
D	Kesilemeyen uyku; çoğunlukla çekirdek I/O bekleyişiyle ilişkilidir.
T / Z	Durdurulmuş / sonlanmış fakat ebeveyni henüz toplamadığı süreç durumu.

Ebeveyn ve çocuk proses ilişkisi

```
ps -o pid,ppid,stat,cmd --forest -u "$USER"
pstree -p "$USER"
```

--forest	ps çıktısında hiyerarşiyi girintili göstermeye yardım eder; destek farklılık gösterebilir.
pstree	Proses ilişkilerini ağaç görünümünde sunar; her sistemde kurulu olmayabilir.
ALTERNATİF	Araç bulunmazsa ps -o pid,ppid,cmd ile başlangıç yapın.
YORUM	Ağaç başlatma ilişkisini gösterir; uygulamanın iç mantığını tek başına açıklamaz.

ps fotoğraf çeker, top değişimi izler

ps	Çalışan süreçleri ve seçilen kullanım istatistiklerini tek bir anda listeler.
top	Etkileşimli ekranı yaklaşık her saniye yenileyerek sistem durumunu ve süreçleri izler.
SIRALAMA	Varsayılan görünümde CPU kullanımına göre en etkin süreçler üstte yer alır.
SORU	“Şu anda kim var?” için ps; “kim etkin ve kullanım nasıl değişiyor?” için top ile başlayın.
SINIR	Tek bir görüntü neden-sonuç açıklamaz; ölçüm aralığı ve diğer kaynakları da değerlendirin.

Top komutu

SPACE	Ekranı hemen yeniler; normalde top yaklaşık saniyede bir yeniler.
P / M / T	CPU kullanımına göre sırala (varsayılan) / yerleşik bellek kullanımına göre sırala / birikimli CPU süresine göre sırala.
u	Süreç listesini tek bir kullanıcıyla sınırlar.
f / ?	Gösterilen istatistik alanlarını seçer / top içindeki kısayolların özetini açar.
DİKKAT	Komut tuşları büyük-küçük harfe duyarlıdır. atop ve htop ek görünüm sunabilir; her sistemde kurulu olmayabilir.

lsof: süreç hangi kaynakları açık tutuyor?

İŞLEV	lsof, açık dosyaları ve bu dosyaları kullanan süreçleri listeler.
DOSYA KAVRAMI	Unix'te bu görünüm normal dosyalarla sınırlı değildir; dizin, ağ kaynağı, dinamik kütüphane ve pipe gibi girdiler de görülebilir.
ÇOK ÇIKTI	Bütün sistemi listelemek uzun ve gürültülü bir çıktı üretir. Önce alan adlarını ve tek bir satırı okuyun.
YETKİ	Normal kullanıcı kendi erişebildiği bilgileri görür; root daha fazlasını görebilir. Bu derste yönetici yetkisi gerektiren işlem yapmayın.

lsof çıktısını satırdan oku

```
# lsof

COMMAND  PID   USER   FD    TYPE    DEVICE  SIZE/OFF    NODE NAME
systemd    1   root  cwd     DIR    8,1      4096          2 /
systemd    1   root  rtd     DIR    8,1      4096          2 /
systemd    1   root  txt     REG    8,1   1595792    9961784
/lib/systemd/systemd
systemd    1   root  mem     REG    8,1   1700792    9961570 /lib/x86_64-linux-
gnu/libm-2.27.so
systemd    1   root  mem     REG    8,1    121016    9961695 /lib/x86_64-linux-
gnu/libudev.so.1

--snip--
vi      1994   juser  cwd     DIR    8,1      4096    4587522 /home/juser
vi      1994   juser   3u     REG    8,1     12288     786440 /tmp/.ff.swp

--snip--
```

COMMAND · PID · USER — Hangi komut, hangi proses ve hangi kullanıcı?

FD: cwd — Bu satır prosesin o anki çalışma dizinini gösterir.

FD: 3u — 3, prosesin kullandığı dosya tanıtıcısının numarasıdır.

TYPE · SIZE/OFF · NODE — Kaynağın türü, boyutu ve inode numarası.

NAME — Açık kaynağın adı/yolu; vi örneği /tmp/.ff.swp dosyasını gösteriyor.

Kaynak: How Linux Works, PDF s.302–303

lsof çıktısını hedefe göre daralt

```
lsof | less
lsof +D /usr
lsof -p "$$"
lsof -h
```

less	Bütün listeyi sayfalı görüntüler; aradığınız adı less içinde arayabilirsiniz.
+D /usr	/usr ağacında açık kaynakları arar; kapsam büyüdükçe yavaşlayabilir.
-p "$$"	Yalnızca bu kabuğun proses kimliğiyle ilişkili açık kaynakları gösterir.
-h	Komut satırı seçeneklerinin kısa yardımını gösterir.

Bir programın ne yaptığını nasıl izleriz?

lsof	Sürecin hangi kaynakları açık tuttuğunu listeler; çoğunlukla mevcut durumu inceler.
strace	Programın çekirdeğe yaptığı sistem çağrılarını izler: dosya açma, okuma, kapatma gibi işlemler.
ltrace	Paylaşılan kütüphanelere yapılan çağrıları izler; statik bağlanmış programlarda çalışmayabilir.
YÖNTEM	İzleme çıktısı çok büyük olabilir. Her satırı ezberlemek yerine hangi çağrıyı ve dönüş değerini aradığınızı belirleyin.

strace: sistem çağrısını ve sonucunu oku

```
strace cat /dev/null
openat(..., "/dev/null", O_RDONLY) = 3
read(3, "", 131072) = 0
close(3) = 0
exit_group(0) = ?
```

openat	Dosyayı açar; 3, çekirdeğin verdiği dosya tanıtıcısıdır.
read	3 numaralı tanıtıcıdan okur; 0 bu örnekte dosya sonunu gösterir.
close	Açık dosya tanıtıcısını kapatır.
exit_group	Sürecin sonlanmasını bildirir. Ayrıntılı çıktı sisteme göre değişebilir.

Hata ayıkla, iz kaydını sakla

```
strace cat not_a_file
openat(..., "not_a_file", O_RDONLY) = -1 ENOENT
strace -o trace.log ./program
strace -o child_trace -ff ./program
strace ./program 2> trace.log
```

ENOENT	-1 açma başarısız; ENOENT “dosya bulunamadı” bilgisidir.
-o	strace çıktısını dosyaya kaydeder; varsayılan hedef standart hatadır.
-ff	Alt süreçleri izler; -o ile her süreç için PID ekli dosya üretir.
2>	Yönlendirme, izlenen komutun standart hata iletilerini de yakalayabilir.

Bir proses, birden çok thread içerebilir

Aynı proses içindeki yürütme akışları ayrı TID taşır; bazı bellek ve kaynakları paylaşır.

TEK PROSES · tek PID · tüm thread'ler aynı prosesin parçası

Ana thread	Thread	Thread
TID = PID	TID ayrı	TID ayrı

PAYLAŞILAN PROSES KAYNAKLARI

Belleğin bir bölümü · açık dosyalar · I/O bağlantıları

Ayrı prosesler ise genellikle kendi adres alanlarını ve kaynaklarını kullanır.

Thread’ler ne zaman yarar sağlar?

EŞZAMANLILIK	Birden çok işlemci varsa thread’ler aynı anda çalışabilir; hesaplama hızlanabilir, ancak bu her program için garanti değildir.
DAHA AZ BAŞLATMA YÜKÜ	Thread başlatmak çoğunlukla yeni proses oluşturmaktan daha hızlıdır.
İLETİŞİM	Aynı prosesin belleğini paylaşmak, thread’ler arası veri paylaşımını çoğu zaman verimli kılar.
ÇOKLU G/Ç	Farklı giriş/çıkış akışlarını yönetmek için yeni proses açmak yerine thread kullanılabilir.

ps m ile PID ve TID ilişkisini gör

```
ps m -o pid,tid,command
12534   -    Xorg
-     12534  -
-     13227  -
```

m	Thread satırlarını proseslerle birlikte listeler.
PID / TID	PID proses kimliği, TID thread kimliğidir. Ana thread'de TID ve PID aynıdır.
TİRE	"-" işareti, o satırda proses veya thread alanının tekrar edilmediğini gösterir.

top içinde thread görünümünü aç

VARSAYILAN	ps ve top normalde yalnızca prosesleri gösterir.
TOP	Büyük H tuşu thread görünümünü açıp kapatır. top içindeki tuşlarda büyük-küçük harfe dikkat edin.
KAYNAK	Bir prosesin thread'leri aynı anda kaynak kullanabilir; görünür thread satırlarını ayrı yürütme akışları olarak yorumlayın.
MÜDAHALE SINIRI	Tek tek thread üzerinde işlem yapmak için programın iç yapısını bilmek gerekir; çoğu durumda bu uygun bir yönetim biçimi değildir.

Kaynak ölçümleri neden değişir?

ZAMAN ARALIĞI	Yüzde kullanım örnekleme aralığına bağlıdır.
ÇEKİRDEKLER	CPU yüzdeleri araç ve ayar tercihine göre bir çekirdek veya toplam kapasiteye göre raporlanabilir.
KISA İŞ	Kısa sürede biten görev, top yenileme aralığında görünmeyebilir.
ÇIKARIM	Düşük CPU kullanımı hata kanıtı değildir; proses I/O, kullanıcı girişi veya zaman bekliyor olabilir.

Kaynak izleme: Ölç, sonra yorumla

KAPSAM	CPU zamanı, bellek ve disk G/Ç hem sistem genelinde hem tek tek süreçler için incelenebilir.
KERNEL	Çekirdek kaynakları süreçlere dağıtır; izleme araçları bu paylaşımın nasıl gerçekleştiğini görmemizi sağlar.
AMAÇ	Yavaşlama varsa önce hangi kaynağın ve hangi sürecin etkili olduğunu araştırın.
AYARLAR	Varsayılan ayarlar çoğu sistemde uygundur. Ölçüm ve gerekçe olmadan performans ayarı değiştirmek anlamlı sonuç vermeyebilir.

Belirli süreçleri top ile izle

KOMUT	top -p PID1 -p PID2
PID	PID1 ve PID2 yer tutucudur; kendi Linux sisteminizde gerçek süreç kimlikleriyle değiştirin.
PS İLE BUL	ps -u "$USER" -o pid,comm komutu kullanıcı süreçlerini ve kimliklerini bulmaya yardım eder.
ZAMAN İÇİNDE	top ekranı yeniler; seçilen süreçlerin kullanımındaki değişimi izleyebilirsiniz.
SINIR	Bir süreç listede görünmüyorsa bitmiş veya erişim kapsamı dışında olabilir.

time: işlemci zamanı ve geçen süre

KABUK KOMUTU	time çoğu kabukta yerleşik olarak bulunur; kapsamlı istatistik sunmayabilir.
DIŞ ARAÇ	/usr/bin/time sistem aracıdır ve daha ayrıntılı istatistik sağlayabilir.
ÖRNEK	time ls komutunu çalıştırın; ls tamamlanınca süre özeti yazdırılır.
ÖRNEK ÇIKTI	real 0m0.442s user 0m0.052s sys 0m0.091s
DİKKAT	Çok kısa komutlarda user ve sys değerleri sıfıra yakın çıkabilir.

real, user ve sys ne anlatır?

USER	CPU'nun programın kendi kodunu çalıştırdığı toplam süre.
SYS	Çekirdeğin program adına yaptığı işlerde kullandığı CPU süresi; örneğin dosya ve dizin işlemleri.
REAL	Komutun başından sonuna kadar geçen duvar saati süresi; CPU başka işler yaparken ve kaynak yanıtı beklerken geçen zaman da buna dahildir.
BEKLEME TAHMİNİ	real − user − sys farkı, sistem veya dış kaynaktan yanıt bekleme süresi hakkında kaba fikir verir; tek başına kesin bir ölçüm değildir.

top çıktısı: PR ve NI alanları

```
$ top
Tasks: 244 total,   2 running, 242 sleeping,   0 stopped,   0 zombie
Cpu(s): 31.7%us,  2.8%sy,  0.0%ni, 65.4%id,  0.2%wa,  0.0%hi,  0.0%si,  0.0%st
Mem:   6137216k total,  5583560k used,   553656k free,    72008k buffers
Swap:  4135932k total,   694192k used,  3441740k free,   767640k cached

  PID USER      PR  NI  VIRT  RES  SHR S %CPU %MEM    TIME+  COMMAND
28883 bri       20   0 1280m 763m  32m S   58 12.7 213:00.65 chromium-browse
 1175 root      20   0  210m  43m  28m R   44  0.7  14292:35 Xorg
 4022 bri       20   0  413m 201m  28m S   29  3.4   3640:13 chromium-browse
 4029 bri       20   0  378m 206m  19m S    2  3.5  32:50.86 chromium-browse
 3971 bri       20   0  881m 359m  32m S    2  6.0 563:06.88 chromium-browse
 5378 bri       20   0  152m  10m 7064 S    1  0.2  24:30.21 xfce4-session
 3821 bri       20   0  312m  37m  14m S    0  0.6  29:25.57 soffice.bin
 4117 bri       20   0  321m 105m  18m S    0  1.8  34:55.01 chromium-browse
 4138 bri       20   0  331m  99m  21m S    0  1.7 121:44.19 chromium-browse
 4274 bri       20   0  232m  60m  13m S    0  1.0  37:33.78 chromium-browse
 4267 bri       20   0 1102m 844m  11m S    0 14.1  29:59.27 chromium-browse
 2327 bri       20   0  301m  43m  16m S    0  0.7 109:55.65 xfce4-panel
```

Kaynak: How Linux Works, PDF s.311

PR

top'un gösterdiği zamanlama önceliği alanı. Tek başına CPU süresini belirlemez.

NI

Nice değeri. Zamanlayıcıya sürecin göreli önceliği hakkında bilgi verir.

ÖRNEK

Görseldeki süreçlerde NI=0. Varsayılan nice değeri 0'dır.

nice değeri CPU paylaşımını nasıl etkiler?

GÖRELİ ETKİ	Nice değeri yükseldikçe süreç, CPU'ya ihtiyaç duyan diğer süreçlere göre daha az öncelikli olur.
ARALIK	Linux'ta nice değeri −20 ile +19 arasındadır. −20 daha yüksek, +19 daha düşük öncelik anlamına gelir; varsayılan değer 0'dır.
SINIR	Nice değeri CPU zamanı garantisi vermez. Çekirdek zamanlayıcısı başka etkenleri de değerlendirir.
YETKİ	Daha düşük öncelik vermek kullanıcıya ait süreçlerde genellikle mümkündür. Daha yüksek öncelik için yönetici yetkisi veya uygun sınırlar gerekir.

renice ile kendi test sürecinin NI değerini değiştir

```
sleep 180 & p=$!
ps -o pid,ni,pri,comm -p "$p"
renice 10 -p "$p"
ps -o pid,ni,pri,comm -p "$p"
kill "$p"
```

TEST SÜRECİ	sleep düşük etkili bir test süreci başlatır; $! PID'yi kaydeder.
RENICE	renice, kendi sürecimizin NI değerini 0'dan 10'a çıkarır.
KONTROL	ps ile önce/sonra NI değerini görün; kill yalnızca bu süreci kapatır.

Load average hangi işleri sayar?

R DURUMU	CPU'da çalışan veya çalışmaya hazır işleri ifade eder.
D DURUMU	Linux load average hesabına kesilemez beklemede olan, çoğunlukla G/Ç bekleyen işler de katılır.
ÖLÇEK	Bu değer, çalışan/hazır işler ile bazı G/Ç bekleyen işlerin zaman içindeki ortalamasıdır.
YÜZDE DEĞİL	Load average yüzde kullanım değildir; D durumundaki G/Ç bekleyişi de değeri yükseltebilir.
UYKU	Kullanıcı girişi veya ağ verisi bekleyen sıradan uyku durumundaki süreçler genellikle bu ortalamaya katılmaz.

uptime: 1, 5 ve 15 dakikalık ortalamalar

KOMUT	uptime sistem çalışma süresini ve yük ortalamalarını gösterir.
ÖRNEK ÇIKTI	load average: 0.08, 0.03, 0.01 · kitapta verilen örnek değerler
1 DAKİKA	En yakın dönem değişimini daha hızlı yansıtır.
5 DAKİKA	Kısa süreli dalgalanmaları biraz daha yumuşatır.
15 DAKİKA	Daha uzun dönem eğilimini gösterir; tek başına anlık durumu açıklamaz.

Load average işlemci sayısıyla birlikte yorumlanır

1 CPU · LOAD 1	Yalnızca çalışabilir işler varsa işlemci kapasitesi yaklaşık doludur.
4 CPU · LOAD 1	Dört işlemcili sistemde bu değer, toplam kapasiteye göre daha düşük rekabete işaret eder.
4 CPU · LOAD 4	Ağırlıkla çalışabilir işler varsa işlemciler ortalama tam doluluğa yakın olabilir.
SINIR	D durumundaki işler de load değerine girer; bu nedenle load / CPU sayısı kesin CPU kullanım yüzdesi değildir.
EĞİLİM	Değerin 1, 5 ve 15 dakikada nasıl değiştiğini ve top'taki süreçleri birlikte inceleyin.

uptime çıktısını sistem kapasitesiyle karşılaştır

```
uptime
nproc
cat /proc/loadavg
```

UPTIME	İlk üç load average değeri 1, 5 ve 15 dakikayı gösterir.
NPROC	Geçerli ortamın kullanabildiği işlemci sayısını gösterir.
PROC DOSYASI	İlk üç alan ortalamalardır; dördüncü alan o an çalışabilir iş / toplam iş sayısıdır.

Yüksek load tek başına sorun anlamına gelmez

BAĞLAM	Load değerini işlemci sayısı, iş yükü ve zaman içindeki eğilimle birlikte değerlendirin.
SİSTEM YANIT VERİYORSA	Bellek ve G/Ç yeterliyse çok sayıda iş CPU'yu paylaşarak çalışabilir; işler yalnızca daha uzun sürede tamamlanır.
SUNUCU İŞ YÜKÜ	Web veya hesaplama sunucularında yüksek yük, beklenen iş yoğunluğuyla ilişkili olabilir.
KISA İŞLER	Çok kısa ömürlü süreçleri tek bir ortalama değeriyle anlamak zordur; araçlar anlık ve zaman ortalamalı farklı görünümler sunar.

Yüksek load ve yavaşlıkta bellek baskısı

OLASI NEDEN	Sistem yavaşlıyor ve load yüksekse olasılıklardan biri bellek baskısıdır; load tek başına nedeni göstermez.
SWAP	Bellek azaldığında sayfaların RAM ile disk arasında taşınması artabilir.
THRASHING	Sistem belleği hızla geri kazanıp taşımaya çalışırken yararlı iş yerine bellek yönetimine çok zaman harcayabilir; yanıt süresi kötüleşir.
KONTROL	CPU, bellek ve G/Ç göstergelerini birlikte inceleyin. Bellek yetersizliği varsaymadan önce ölçün.
GEÇİŞ	Sonraki içerikte bellek durumunu ve bellek kaynaklı yavaşlamayı ele alacağız.

free ve /proc/meminfo ile belleği gözlemle

```
free -h
less /proc/meminfo
```

FREE	Fiziksel bellek ve swap için kısa özet verir.
MEMINFO	Sistem belleği alanlarını ayrıntılı listeler.
CACHE/BUF	Önbelleğin bir bölümü ihtiyaç halinde geri kazanılabilir.
AVAILABLE	Yeni işler için tahmini alanı gösterir; yalnız free ile tanı koymayın.
ORTAM	Linux VM veya laboratuvarda çalıştırın; macOS'ta bu araçlar yoktur.

Sanal adres, MMU ve fiziksel bellek

SAYFA	Çekirdek, bellek adres alanını sabit boyutlu sayfalara böler.
SÜRECİN GÖRÜNÜMÜ	Program sanal adreslerle çalışır; fiziksel RAM adreslerini doğrudan kullanmaz.
SAYFA TABLOSU	Çekirdeğin yönettiği eşleme yapısı sanal sayfaları fiziksel bellek sayfalarıyla ilişkilendirir.
MMU	İşlemcideki bellek yönetim birimi, bellek erişiminde sanal adresi sayfa tablosuna göre fiziksel adrese çevirir.

Demand paging: sayfalar gerektiğinde yüklenir

BAŞLANGIÇ	Çekirdek program kodunun başlangıcını ve bazı çalışma sayfalarını belleğe alır.
İLK ERİŞİM	Program, bellekte henüz bulunmayan geçerli bir sayfaya erişebilir.
PAGE FAULT	MMU bu durumu çekirdeğe bildirir; page fault burada olağan bir bellek yönetimi olayıdır, tek başına program hatası değildir.
SAYFAYI YÜKLE	Çekirdek gerekli sayfayı bulur veya bellekte yer açar, eşlemeyi günceller ve programı kaldığı yerden sürdürür.
AMAÇ	Programın bütün sayfalarının başlangıçta RAM'de bulunması gerekmez; ihtiyaç oldukça yükleme yapılır.

Sistemin sayfa boyutunu öğren

```
getconf PAGE_SIZE
```

ÇIKTI	Örnek Linux sisteminde 4096 bayt, yani 4 KiB.
YORUM	Bu değer sayfa boyutudur; bütün Linux sistemlerinde aynı olmak zorunda değildir.
UYGULAMA	Bellek konularında sabit bir sayı varsaymak yerine kendi Linux ortamınızda komutu çalıştırın.

Page fault: bellek erişimi çekirdeğe geçiş başlatır

OLAY	Süreç istediği sayfaya doğrudan erişemediğinde CPU çekirdeğe geçer.
ÇEKİRDEĞİN KARARI	Adres geçerliyse çekirdek sayfayı bulur veya hazırlar, eşlemeyi kurar ve süreci sürdürür.
HER FAULT HATA DEĞİL	Demand paging sırasında page fault olağan bir mekanizmadır; tek başına programın çöktüğünü göstermez.
GEÇERSİZ ADRES	Adres sürece ait değilse çekirdek isteği karşılayamaz; süreç hata sinyali alabilir.
BAĞLANTI	Bir önceki slaytta gördüğümüz demand paging, geçerli ama henüz hazır olmayan sayfanın yüklenmesini açıklar.

vmstat sistem kaynaklarını tek tabloda toplar

PROCS	r çalışmaya hazır kuyruk; b kesilemeyen bekleme durumundaki işler.
MEMORY	swpd kullanılan swap alanını; free, buff ve cache bellek değerlerini gösterir.
SWAP	si belleğe alınan, so diske çıkarılan sayfaların etkinliğini gösterir.
I/O	bi/bo blok aygıtlarından okuma ve yazma hareketini gösterir.
SYSTEM	in kesmeler, cs bağlam değiştirmeleri için saniyedeki sayıdır.
CPU	us kullanıcı kodu, sy çekirdek, id boş ve wa G/Ç bekleme süresidir.

Sistem sakinken vmstat çıktısı

```
$ vmstat 2
procs -----------memory---------- ---swap-- -----io---- -system-- ----cpu----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa
 2  0 320416 3027696 198636 1072568    0    0     1     1    2    0 15  2 83  0
 2  0 320416 3027288 198636 1072564    0    0     0  1182  407  636  1  0 99  0
 1  0 320416 3026792 198640 1072572    0    0     0    58  281  537  1  0 99  0
 0  0 320416 3024932 198648 1074924    0    0     0   308  318  541  0  0 99  1
 0  0 320416 3024932 198648 1074968    0    0     0     0  208  416  0  0 99  0
 0  0 320416 3026800 198648 1072616    0    0     0     0  207  389  0  0 100  0
```

SWAP swpd dolu olsa da si=0 ve so=0: ölçüm aralığında swap aktarımı yok.

CPU Sonraki örneklerde id yüksek: CPU çoğunlukla boşta; vmstat 2 iki saniyede bir günceller.

Bir program başlarken bellek ve I/O hareketi

```
procs -----------memory---------- ---swap-- -----io---- -system-- ----cpu----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa
 1  0 320412 2861252 198920 1106804    0    0     0     0 2477 4481 25  2 72  0
❶
 1  0 320412 2861748 198924 1105624    0    0     0    40 2206 3966 26  2 72  0
 1  0 320412 2860508 199320 1106504    0    0   210    18 2201 3904 26  2 71  1
 1  1 320412 2817860 199332 1146052    0    0 19912     0 2446 4223 26  3 63  8
 2  2 34 2791608 200612 1157752  202    0  4960   854 3371 5714 27  3 51 18 ❷
 1  1 320252 2772076 201076 1166656   10    0  2142  1190 4188 7537 30  3 53 14
 0  3 320244 2727632 202104 1175420   20    0  1890   216 4631 8706 36  4 46 14
```

① CPU işi artarken cache/buff büyür; sistemde hâlâ boş RAM vardır.

② si swap-in, b bekleyen işlerdir; bi/bo ve wa G/Ç etkinliğini gösterir. Tek örnek thrashing kanıtı değildir.

vmstat ölçümlerini doğru zaman aralığında oku

```
vmstat 2 5
```

İLK SATIR	Başlangıçtan beri ortalamadır; teşhiste sonraki satırlara bakın.
SONRAKİLER	İki saniyelik aralıklarla ölçüm verir; 5 satır sonra durur.
SWAP	si/so sıfırsa aktarım yoktur; swpd yine dolu olabilir.
BAĞLAM	Süren si/so, b, disk I/O, wa ve yavaşlık birlikteyse araştırın.
SINIR	Birim ve alanlar sürüme göre değişebilir; vmstat(8) ile doğrulayın.

I/O incelemesinde ölçüm düzeyini seç

VMSTAT	vmstat -d ayrıntılı disk/partition verisi; genel görünümde CPU ve belleği de verir.
IOSTAT	Depolama aygıtı ve bölüm düzeyinde okuma/yazma ölçümleri sağlar.
IOTOP	Anlık I/O etkinliğini süreç/thread satırlarıyla gösterir.
PIDSTAT	Belirli bir PID'nin CPU, bellek veya disk kullanımını zaman içinde izler.
KURULUM	Araçlar her Linux dağıtımında hazır olmayabilir; yetkili VM/lab ortamını kullanın.

iostat aygıt düzeyinde I/O ölçer

```
$ iostat
[kernel information]
avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           4.46    0.01    0.67    0.31    0.00   94.55

Device:            tps    kB_read/s    kB_wrtn/s    kB_read    kB_wrtn
sda               4.67         7.28        49.86    9493727   65011716
sde               0.00         0.00         0.00       1230          0
```

AVG-CPU	Üst bölüm CPU yüzdeleridir; aygıt tablosuna odaklanın.
TPS / RATE	tps aktarım/s; read/s ve wrtn/s saniyelik veri miktarıdır.
ZAMAN	Varsayılan çıktı açılıştan beri; iostat -d 2 aygıt raporunu aralıkla günceller.

iostat -p ALL disk bölümlerini de gösterir

```
$ iostat -p ALL
--snip--
Device:            tps    kB_read/s    kB_wrtn/s    kB_read    kB_wrtn
--snip--
sda               4.67         7.27        49.83    9496139   65051472
sda1              4.38         7.16        49.51    9352969   64635440
sda2              0.00         0.00         0.00          6          0
sda5              0.01         0.11         0.32     141884     416032
scd0              0.00         0.00         0.00          0          0
--snip--
sde               0.00         0.00         0.00       1230          0
```

iotop etkin I/O kullanan thread'leri gösterir

```
# iotop
Total DISK READ:       4.76 K/s | Total DISK WRITE:     333.31 K/s
  TID  PRIO  USER     DISK READ  DISK WRITE  SWAPIN     IO>    COMMAND
  260 be/3 root        0.00 B/s   38.09 K/s  0.00 %  6.98 % [jbd2/sda1-8]
 2611 be/4 juser       4.76 K/s   10.32 K/s  0.00 %  0.21 % zeitgeist-daemon
 2636 be/4 juser       0.00 B/s   84.12 K/s  0.00 %  0.20 % zeitgeist-fts
 1329 be/4 juser       0.00 B/s   65.87 K/s  0.00 %  0.03 % soffice.b~ash-pipe=6
 6845 be/4 juser       0.00 B/s  812.63 B/s  0.00 %  0.00 % chromium-browser
19069 be/4 juser       0.00 B/s  812.63 B/s  0.00 %  0.00 % rhythmbox
```

TID Satırlar PID değil thread kimliği gösterir; önceki thread konusuyla bağlantı kurun.

GÖRÜNÜM Üstte toplam disk okuma/yazma, satırlarda thread başına etkinlik sürekli güncellenir.

pidstat tek sürecin kullanımını zaman içinde gösterir

```
$ pidstat -p 1329 1
Linux 5.4.0-48-generic (duplex)         11/09/2020      _x86_64_        (4 CPU)

09:26:55 PM   UID  PID    %usr %system  %guest    %CPU   CPU  Command
09:27:03 PM  1000  1329    8.00   0.00    0.00    8.00     1  myprocess
09:27:04 PM  1000  1329    0.00   0.00    0.00    0.00     3  myprocess
09:27:05 PM  1000  1329    3.00   0.00    0.00    3.00     1  myprocess
09:27:06 PM  1000  1329    8.00   0.00    0.00    8.00     3  myprocess
09:27:07 PM  1000  1329    2.00   0.00    0.00    2.00     3  myprocess
09:27:08 PM  1000  1329    6.00   0.00    0.00    6.00     2  myprocess
```

ÖRNEK pidstat -p 1329 1: her satır yaklaşık bir saniyelik örnektir.

CPU %usr, %system, %CPU ve prosesin çalıştığı CPU numarası görünür.

pidstat ile CPU, bellek ve disk ölçümü

```
PID=12345
pidstat -p "$PID" 1 5
pidstat -r -p "$PID" 1 5
pidstat -d -p "$PID" 1 5
```

CPU	Varsayılan görünüm CPU kullanımını örnekler.
-r	Bellek kullanımına ilişkin alanları ekler.
-d	Prosesin disk I/O ölçümlerini gösterir.
PARAMETRE	1 saniye arayla 5 örnek alınır; 12345 yerine gerçek PID'nizi yazın.

Linux’ta Süreç ve Kaynak İzleme

Süreçleri gözlemleme ve CPU, bellek, I/O kullanımı